📘 Building a Data Warehouse Using the 3-Layer Architecture

🏗️ Introduction

In this project, we will design and implement a Data Warehouse using the three-layer architecture: Bronze, Silver, and Gold. The goal is to transform raw transactional data into structured, high-quality, and analytics-ready data using PostgreSQL or any SQL-based database.

The first step is to visualize the architecture with a Data Flow Diagram 📊. Below is a representation of how data flows from the source systems (CRM & ERP) through the different layers before reaching the final analytics-ready format (Gold Layer).

📌 Step-by-Step Process of Building the Data Warehouse

Step 1: Bronze Layer (Raw Data Ingestion) 🏛️

🔷 Purpose: Store raw data without transformations to preserve its original state.

🔷 Action: Extract data from CRM & ERP sources and store it as-is in the Bronze Layer.

🔷 Tables:

- crm_sales_details
- crm_cust_info
- crm_prd_info

Step 2: Silver Layer (Data Cleaning & Integration) ⚙️

🔷 Purpose: Apply data cleansing, deduplication, filtering, and standardization to ensure data quality.

🔷 Action: Transform and integrate data from the Bronze Layer by applying business rules.

🔷 Tables:

- crm_sales_details (Cleaned)
- crm_cust_info (Enriched)
- crm_prd_info (Standardized)

Step 3: Gold Layer (Analytics-Ready Data) 🌟

🔷 Purpose: Create optimized data models (Fact & Dimension tables) for reporting and analysis.

🔷 Action:

- Join transactional data with dimension tables (customers & products).
- Create views to structure the data into Facts & Dimensions.

🔷 Tables:

- fact_sales (Sales Transaction Data)
- dim_customers (Customer Dimension)
- dim_products (Product Dimension)

📌 Final Data Model Overview

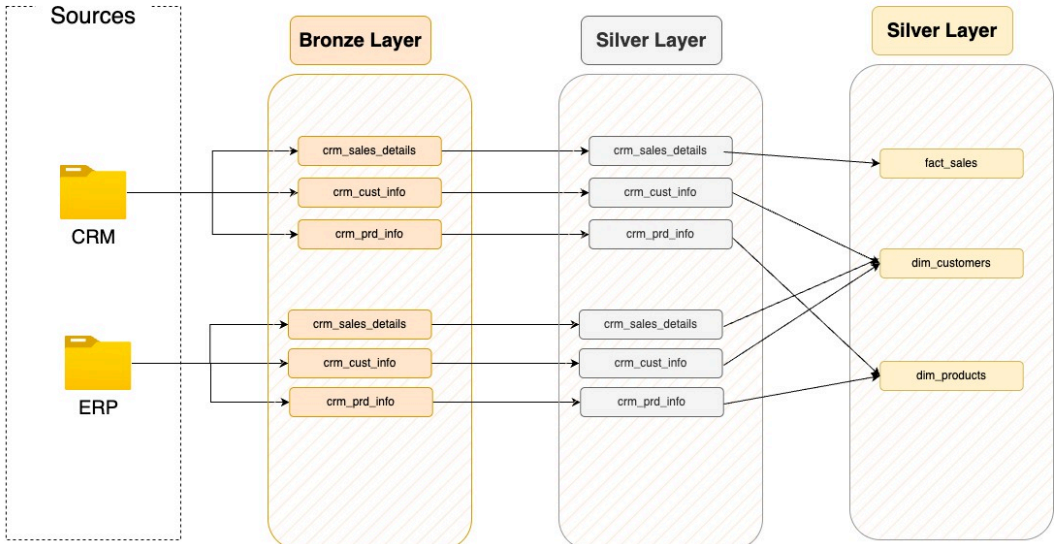
📊 The Gold Layer now holds clean, structured, and analytics-ready data that can be used for reporting, dashboards, and advanced analytics. This three-layer approach ensures scalability, flexibility, and data consistency across the system.

🚀 Next Steps:
Once the data warehouse is set up, we can integrate BI tools (Power BI, Tableau, or Looker) to generate powerful insights! 📈

✅ Conclusion

By following this structured 3-layer approach, we can ensure that our Data Warehouse is organized, scalable, and ready for business analytics. Now, let's move forward with writing SQL

Data Flow Diagram



(scripts to implement this architecture in PostgreSQL! 🔥)

```
-- 🏛️ BRONZE LAYER: RAW DATA STORAGE
-- --------------------------------
-- The Bronze Layer stores raw, unprocessed data from various source systems
-- (CRM, ERP, etc.) before any transformations or cleaning.
-- This ensures we always have an original copy of the data.

-- 📌 Key Features of the Bronze Layer:
-- ✅ Stores data as-is from source systems.
-- ✅ Provides a structured schema for organizing data.
-- ✅ Acts as a foundation for further transformation and analytics.
-- ✅ Useful for historical tracking and auditing.

-- 🔄 STEP 1: Create Tables (Drop if they already exist to avoid duplication)
-- ------------------------------------------------------------------------

-- 📌 CRM Customer Information Table
-- --------------------------------
DROP TABLE IF EXISTS bronze.crm_cust_info;
CREATE TABLE bronze.crm_cust_info (
    cst_id INT,                    -- Unique Customer ID
    cst_key VARCHAR(50),           -- Unique Customer Key
    cst_firstname VARCHAR(50),     -- First Name
    cst_lastname VARCHAR(50),      -- Last Name
    cst_marital_status VARCHAR(50),  -- Marital Status (Single, Married, etc.)
    cst_gndr VARCHAR(50),          -- Gender
    cst_create_date DATE           -- Account Creation Date
);
-- 🔍 Verify Table
SELECT * FROM bronze.crm_cust_info LIMIT 10;
-- 🔄 Reset Data if Needed
-- TRUNCATE TABLE bronze.crm_cust_info;

-- 📌 CRM Product Information Table
-- --------------------------------
DROP TABLE IF EXISTS bronze.crm_prd_info;
CREATE TABLE bronze.crm_prd_info (
    prd_id INT,                    -- Unique Product ID
    prd_key VARCHAR(50),           -- Unique Product Key
    prd_nm VARCHAR(50),            -- Product Name
    prd_cost INT,                  -- Product Cost
    prd_line VARCHAR(50),          -- Product Line (e.g., Electronics, Furniture)
    prd_start_dt TIMESTAMP,        -- Availability Start Date
    prd_end_dt TIMESTAMP           -- Availability End Date
);
-- 🔍 Verify Table
SELECT * FROM bronze.crm_prd_info LIMIT 10;
-- 🔄 Reset Data if Needed
-- TRUNCATE TABLE bronze.crm_prd_info;

-- 📌 CRM Sales Details Table
-- --------------------------
DROP TABLE IF EXISTS bronze.crm_sales_details;
CREATE TABLE bronze.crm_sales_details (
```

```
    sls_ord_num VARCHAR(50),          -- Sales Order Number
    sls_prd_key VARCHAR(50),          -- Product Key (Linked to crm_prd_info)
    sls_cust_id INT,                  -- Customer ID (Linked to crm_cust_info)
    sls_order_dt INT,                  -- Order Date (Consider converting to TIMESTAMP)
    sls_ship_dt INT,                  -- Shipping Date
    sls_due_dt INT,                   -- Due Date for Payment
    sls_sales INT,                    -- Total Sales Amount
    sls_quantity INT,                 -- Quantity Sold
    sls_price INT                     -- Product Price at the Time of Sale
);
-- 🔍 Verify Table
SELECT * FROM bronze.crm_sales_details LIMIT 10;
-- 🔄 Reset Data if Needed
-- TRUNCATE TABLE bronze.crm_sales_details;

-- 📌 ERP Location Data Table
-- ---------------------------
DROP TABLE IF EXISTS bronze.erp_loc_a101;
CREATE TABLE bronze.erp_loc_a101 (
    cid VARCHAR(50),                  -- Customer ID
    cntry VARCHAR(50)                 -- Country Code
);
-- 🔍 Verify Table
SELECT * FROM bronze.erp_loc_a101 LIMIT 10;
-- 🔄 Reset Data if Needed
-- TRUNCATE TABLE bronze.erp_loc_a101;

-- 📌 ERP Customer Demographics Table
-- ----------------------------------
DROP TABLE IF EXISTS bronze.erp_cust_az12;
CREATE TABLE bronze.erp_cust_az12 (
    cid VARCHAR(50),                  -- Customer ID
    bdate DATE,                       -- Birthdate
    gen VARCHAR(50)                   -- Gender
);
-- 🔍 Verify Table
SELECT * FROM bronze.erp_cust_az12 LIMIT 10;
-- 🔄 Reset Data if Needed
-- TRUNCATE TABLE bronze.erp_cust_az12;

-- 📌 ERP Product Category Table
-- -----------------------------
DROP TABLE IF EXISTS bronze.erp_px_cat_g1v2;
CREATE TABLE bronze.erp_px_cat_g1v2 (
    id VARCHAR(50),                   -- Product ID
    cat VARCHAR(50),                  -- Product Category (e.g., Electronics, Clothing)
    subcat VARCHAR(50),               -- Product Subcategory (e.g., Laptops, Shoes)
    maintenance VARCHAR(50)           -- Maintenance Type (if applicable)
);
-- 🔍 Verify Table
SELECT * FROM bronze.erp_px_cat_g1v2 LIMIT 10;
-- 🔄 Reset Data if Needed
-- TRUNCATE TABLE bronze.erp_px_cat_g1v2;
```

```
-- ✅ FINAL CHECK: Ensure all tables exist
-- ---------------------------------------
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'bronze';




-- 🏛️ SILVER LAYER: CLEANED & TRANSFORMED DATA STORAGE
-- ---------------------------------------------------
-- 🎯 **Goal:** Move cleaned customer data from the Bronze layer to the Silver layer.
-- 🔷 **Steps Covered in This Script:**
--     1️⃣ Check for NULL values and duplicates in the Bronze layer.
--     2️⃣ Select the most recent and complete row per customer.
--     3️⃣ Standardize and clean the data (trimming spaces, mapping values).
--     4️⃣ Insert transformed data into the Silver layer.
--     5️⃣ Verify that the new Silver table does not contain NULL values.

-- 🟢 STEP 1: CHECK FOR DUPLICATES IN PRIMARY KEY (cst_id)
-- -------------------------------------------------------
-- 🎯 **Why?** The `cst_id` column should be unique. If duplicates exist, we must resolve them.
-- ✅ **Expectation:** This query should return **0 rows** (i.e., no duplicate `cst_id` values).

SELECT
    cst_id,
    COUNT(*)
FROM bronze.crm_cust_info
GROUP BY cst_id
HAVING COUNT(*) > 1;  -- 🚨 If this returns rows, it means duplicates exist.

-- 🟢 STEP 2: REMOVE DUPLICATES BY SELECTING THE MOST RECENT RECORD
-- ----------------------------------------------------------------
-- 🎯 **Why?** If a customer has multiple records, keep only the latest entry.
-- 🔍 **How?** Use `ROW_NUMBER()` to select the **most recent** `cst_create_date`.
-- ✅ **Expectation:** Each `cst_id` will have only **one row** in the result.

SELECT
    *
FROM (
    SELECT
        *,
        ROW_NUMBER() OVER (PARTITION BY cst_id ORDER BY cst_create_date DESC) AS
flag_last
    FROM bronze.crm_cust_info
) subquery
WHERE flag_last = 1;  -- ✅ Keeps only the most recent record for each customer.
```

```
-- 🟢 STEP 3: CHECK DUPLICATE ENTRIES (EXTRA RECORDS)
-- ------------------------------------------------
-- 🎯 **Why?** To verify how many duplicate records exist in the dataset.
-- 🔍 **How?** If a customer has more than **one row**, the extra rows are flagged.
-- ✅ **Expectation:** If there are **duplicates**, this query will return them.

SELECT
    *
FROM (
    SELECT
        *,
        ROW_NUMBER() OVER (PARTITION BY cst_id ORDER BY cst_create_date DESC) AS
flag_last
    FROM bronze.crm_cust_info
) subquery
WHERE flag_last != 1;  -- 🚨 These are **duplicate records** that we are NOT inserting.

-- 🟢 STEP 4: CLEAN & TRANSFORM DATA BEFORE INSERTING INTO SILVER LAYER
-- ------------------------------------------------------------------
-- 🎯 **Why?**
-- 🔷 1️⃣ Trim unwanted spaces.
-- 🔷 2️⃣ Standardize values (e.g., 'S' → 'Single', 'M' → 'Married').
-- 🔷 3️⃣ Ensure gender values match database constraints.
-- 🔍 **How?** Use `TRIM()`, `CASE`, and transformations.

SELECT
    cst_id,
    cst_key,
    TRIM(cst_firstname) AS cst_firstname,  -- 🧹 Removes leading/trailing spaces.
    TRIM(cst_lastname) AS cst_lastname,    -- 🧹 Removes leading/trailing spaces.
    CASE
        WHEN cst_marital_status = 'S' THEN 'Single'
        WHEN cst_marital_status = 'M' THEN 'Married'
        ELSE 'n/a'  -- 🔍 Standardizing marital status.
    END AS cst_marital_status,
    CASE
        WHEN cst_gndr = 'F' THEN 'Female'
        WHEN cst_gndr = 'M' THEN 'Male'
        ELSE 'other'  -- 🔍 Fixing gender values to match CHECK constraints.
    END AS cst_gndr,
    cst_create_date
FROM (
    SELECT
        *,
        ROW_NUMBER() OVER (PARTITION BY cst_id ORDER BY cst_create_date DESC) AS
flag_last
    FROM bronze.crm_cust_info
) subquery
WHERE flag_last = 1;  -- ✅ Ensures only one (most recent) row per customer.

-- 🟢 STEP 5: INSERT CLEAN DATA INTO SILVER LAYER TABLE
```

```
-- ------------------------------------------------
-- 🎯 **Why?** This moves the cleaned and validated data into the Silver layer.
-- 🔷 **Ensures:**
--     ✅ No duplicates.
--     ✅ No NULL primary keys.
--     ✅ Standardized formats.
--     ✅ Tracks insertion timestamp (`dwh_create_date`).
TRUNCATE TABLE silver.crm_cust_info
INSERT INTO silver.crm_cust_info (
    cst_id,
    cst_key,
    cst_firstname,
    cst_lastname,
    cst_marital_status,
    cst_gndr,
    cst_create_date,
    dwh_create_date
)
SELECT
    cst_id,
    cst_key,
    TRIM(cst_firstname) AS cst_firstname,
    TRIM(cst_lastname) AS cst_lastname,
    CASE
        WHEN cst_marital_status = 'S' THEN 'Single'
        WHEN cst_marital_status = 'M' THEN 'Married'
        ELSE 'n/a'
    END AS cst_marital_status,
    CASE
        WHEN cst_gndr = 'F' THEN 'Female'
        WHEN cst_gndr = 'M' THEN 'Male'
        ELSE cst_gndr  -- ✅ Now matches CHECK constraint
    END AS 'other',
    cst_create_date,
    CURRENT_TIMESTAMP  -- 🕒 Records when the row was inserted into the Silver layer.
FROM (
    SELECT
        *,
        ROW_NUMBER() OVER (PARTITION BY cst_id ORDER BY cst_create_date DESC) AS
flag_last
    FROM bronze.crm_cust_info
) subquery
WHERE flag_last = 1
AND cst_id IS NOT NULL;  -- ✅ Excludes rows where cst_id is NULL.

-- 🟢 STEP 6: FINAL VALIDATION - CHECK FOR NULL VALUES IN THE SILVER TABLE
-- ------------------------------------------------------------------------
-- 🎯 **Why?** We need to ensure that all required columns have valid data.
-- ✅ **Expectation:** This query should return **0 rows** (i.e., no NULLs exist).

SELECT * FROM silver.crm_cust_info
WHERE cst_id IS NULL
   OR cst_key IS NULL
   OR cst_firstname IS NULL
```

```
    OR cst_lastname IS NULL
    OR cst_marital_status IS NULL
    OR cst_gndr IS NULL
    OR cst_create_date IS NULL
    OR dwh_create_date IS NULL;  -- 🚨 If this query returns rows, we have NULL values.

/*
===============================================================================
================
🏛️ SILVER LAYER - FINAL SUMMARY 🏛️
===============================================================================
================
🎯 **Goal:** Ensure clean, unique, and well-structured data in `silver.crm_cust_info`.
🔍 **Below are the key steps, their purpose, and the expected outcome:**

┌──────────┬──────────────────────────┬──────────────────────────────────────┐
| 🏷️ Step | 🎯 Purpose               | ✅ Expected Outcome                  |
├──────────┼──────────────────────────┼──────────────────────────────────────┤
| 1️⃣       | Check for Duplicates     | No duplicate `cst_id` values         |
|          | (Primary Key Validation) | **(Query should return 0 rows)**     |
├──────────┼──────────────────────────┼──────────────────────────────────────┤
| 2️⃣       | Select the Latest Record | Each customer has **only 1 row**     |
|          | (Most recent `cst_create_date`) | Duplicates are removed        |
├──────────┼──────────────────────────┼──────────────────────────────────────┤
| 3️⃣       | Identify Extra Duplicates| Returns extra rows that exist        |
|          | (For validation only)    | Helps in debugging                   |
├──────────┼──────────────────────────┼──────────────────────────────────────┤
| 4️⃣       | Clean & Standardize Data | No leading/trailing spaces           |
|          | (`TRIM()`, CASE mappings)| Standardized values                  |
├──────────┼──────────────────────────┼──────────────────────────────────────┤
| 5️⃣       | Insert into Silver Table | Only **clean & latest** records      |
|          | (Insert transformed data)| Data is **ready for analysis**       |
├──────────┼──────────────────────────┼──────────────────────────────────────┤
| 6️⃣       | Validate for NULLs       | **No NULL values should exist**      |
|          | (`NOT NULL` constraint)  | Ensures data integrity               |
└──────────┴──────────────────────────┴──────────────────────────────────────┘

✅ **Final Check:** After executing all steps, the Silver Layer should have:
```

- Clean, unique, and **standardized data**
- No duplicates or **NULL values**
- **Correctly formatted** fields for reporting and analytics

🎯 **Next Step:** Move validated data to the **Gold Layer** for Business Intelligence processing.

==========
*/

/*
==========
🏛️ **DATA PROCESSING: CRM PRODUCT INFORMATION (`crm_prd_info`)**
==========
🎯 **Goal:** Extract, clean, and transform data from `bronze.crm_prd_info` into `silver.crm_prd_info` while ensuring data consistency.

📌 **Steps Overview:**
1️⃣ **Check for duplicates and NULL values** in `prd_id`
2️⃣ **Check for unwanted spaces in `prd_nm`**
3️⃣ **Validate `prd_cost` values (non-negative and not NULL)**
4️⃣ **Transform and clean data before insertion**
5️⃣ **Create the `silver.crm_prd_info` table**
6️⃣ **Insert cleaned data into the Silver table**
7️⃣ **Verify the transformation results**
==========
*/

-- 🟢 STEP 1: CHECK FOR DUPLICATES AND NULL VALUES IN `prd_id`
-- ----------
-- 🎯 **Why?** `prd_id` should be unique and not NULL.
-- ✅ **Expectation:** Query should return **0 rows** if no issues exist.

```
SELECT
    prd_id,
    COUNT(*)
FROM bronze.crm_prd_info
GROUP BY prd_id
HAVING COUNT(*) > 1 OR prd_id IS NULL;
```

-- 🟢 STEP 2: CHECK FOR UNWANTED SPACES IN `prd_nm`
-- ----------
-- 🎯 **Why?** Ensures consistency in `prd_nm` values by trimming spaces.
-- ✅ **Expectation:** Query should return **0 rows** if all names are correctly formatted.

```
SELECT prd_nm
```

```
FROM bronze.crm_prd_info
WHERE prd_nm != TRIM(prd_nm);

-- 🟢 STEP 3: VALIDATE `prd_cost` COLUMN (NON-NEGATIVE & NOT NULL)
-- ---------------------------------------------------------------
-- 🎯 **Why?** Product cost should never be negative or NULL.
-- ✅ **Expectation:** Query should return **0 rows** if values are valid.

SELECT
    prd_cost
FROM bronze.crm_prd_info
WHERE prd_cost < 0 OR prd_cost IS NULL;

-- 🟢 STEP 4: TRANSFORM & CLEAN DATA BEFORE INSERTING INTO SILVER TABLE
-- ---------------------------------------------------------------------
-- 🎯 **Why?**
-- 🔷 **Standardize category ID (`cat_id`)** by replacing `-` with `_` in `prd_key`
-- 🔷 **Extract main product key (`prd_key`)** by removing the first 6 characters
-- 🔷 **Ensure `prd_cost` is never NULL** (replace with `0` using `COALESCE()`)
-- 🔷 **Map short product line codes (`M`, `R`, `S`, `T`)** to full names
-- 🔷 **Convert `prd_start_dt` & `prd_end_dt` to DATE format**

SELECT
    prd_id,
    prd_key,
    REPLACE(SUBSTRING(prd_key, 1, 5), '-','_') AS cat_id,  -- 🔷 Standardized category ID
    SUBSTRING(prd_key FROM 7 FOR LENGTH(prd_key)) AS prd_key,  -- 🔷 Extract main product
key
    prd_nm,
    COALESCE(prd_cost, 0) AS prd_cost,  -- 🔷 Ensure cost is never NULL
    CASE
        WHEN UPPER(TRIM(prd_line)) = 'M' THEN 'Mountain'
        WHEN UPPER(TRIM(prd_line)) = 'R' THEN 'Road'
        WHEN UPPER(TRIM(prd_line)) = 'S' THEN 'Other Sales'
        WHEN UPPER(TRIM(prd_line)) = 'T' THEN 'Touring'
        ELSE 'n/a'
    END AS prd_line,  -- 🔷 Standardized product line
    CAST(prd_start_dt AS DATE) AS prd_start_dt,  -- 🔷 Convert to DATE format
    CAST(LEAD(prd_start_dt) OVER (PARTITION BY prd_key ORDER BY prd_start_dt - INTERVAL
'1 day') AS DATE) AS prd_end_dt
    -- 🔷 Assigns the next product's start date as this product's end date
FROM bronze.crm_prd_info;

-- 🟢 STEP 5: CREATE `silver.crm_prd_info` TABLE
-- ---------------------------------------------
-- 🎯 **Why?** Create the Silver Layer table without changing your schema.

DROP TABLE IF EXISTS silver.crm_prd_info;
CREATE TABLE silver.crm_prd_info (
    prd_id INT,
    cat_id VARCHAR(50),
```

```
    prd_key VARCHAR(50),
    prd_nm VARCHAR(50),
    prd_cost INT,
    prd_line VARCHAR(50),
    prd_start_dt DATE,
    prd_end_dt DATE,
    dwh_create_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 🟢 STEP 6: INSERT CLEANED DATA INTO THE SILVER TABLE
-- ------------------------------------------------
-- 🎯 **Why?** Load transformed data into the Silver Layer.
TRUNCATE TABLE silver.crm_prd_info
INSERT INTO silver.crm_prd_info (
    prd_id,
    cat_id,
    prd_key,
    prd_nm,
    prd_cost,
    prd_line,
    prd_start_dt,
    prd_end_dt
)
SELECT
    prd_id,
    REPLACE(SUBSTRING(prd_key, 1, 5), '-','_') AS cat_id,
    SUBSTRING(prd_key FROM 7 FOR LENGTH(prd_key)) AS prd_key,
    prd_nm,
    COALESCE(prd_cost, 0) AS prd_cost,
    CASE
        WHEN UPPER(TRIM(prd_line)) = 'M' THEN 'Mountain'
        WHEN UPPER(TRIM(prd_line)) = 'R' THEN 'Road'
        WHEN UPPER(TRIM(prd_line)) = 'S' THEN 'Other Sales'
        WHEN UPPER(TRIM(prd_line)) = 'T' THEN 'Touring'
        ELSE 'n/a'
    END AS prd_line,
    CAST(prd_start_dt AS DATE) AS prd_start_dt,
    CAST(LEAD(prd_start_dt) OVER (PARTITION BY prd_key ORDER BY prd_start_dt - INTERVAL
'1 day') AS DATE) AS prd_end_dt
FROM bronze.crm_prd_info;

-- 🟢 STEP 7: VALIDATE DATA INSERTION
-- ---------------------------------
-- 🎯 **Why?** Ensure that data has been successfully inserted and transformed.

SELECT * FROM silver.crm_prd_info;

/*
==========================================================================
===============
🏛️ **FINAL SUMMARY: CRM PRODUCT INFORMATION PROCESSING (`crm_prd_info`)**
==========================================================================
===============

🎯 **Goal:** Ensure data quality, transformation, and insertion into `silver.crm_prd_info`.
```

Step	Action	Expected Outcome
1️⃣	Check duplicates & NULLs	No duplicate or NULL `prd_id` values
2️⃣	Validate product names	No extra spaces in `prd_nm`
3️⃣	Check cost values	No negative or NULL `prd_cost`
4️⃣	Transform & clean data	Standardized & formatted product data
5️⃣	Create Silver table	Table structure matches requirements
6️⃣	Insert cleaned data	Data is loaded into Silver Layer
7️⃣	Validate inserted data	Query returns correctly transformed data

✅ **Final Check:** After executing all steps, the Silver Layer should have:
- Clean, unique, and **standardized data**
- No duplicates or **NULL values**
- **Correctly formatted** fields for reporting and analytics

🎯 **Next Step:** Move validated data to the **Gold Layer** for Business Intelligence processing.

==========
*/

/*
==========
🏛️ **SALES DETAILS DATA PROCESSING (`crm_sales_details`)**
==========

🎯 **Goal:**
✅ Extract, clean, and transform sales data from `bronze.crm_sales_details`.
✅ Ensure **data integrity** by handling **NULL values**, **incorrect dates**, and **price mismatches**.
✅ Load the cleaned data into `silver.crm_sales_details` for **further analytics and reporting**.

📌 **Steps Overview:**
1️⃣ **Check for NULL values and incorrect data**
2️⃣ **Ensure date fields are in the correct format**
3️⃣ **Validate sales consistency (`sls_sales = sls_quantity * sls_price`)**
4️⃣ **Create `silver.crm_sales_details` table**
5️⃣ **Insert cleaned and transformed data into the Silver table**
6️⃣ **Verify the inserted data**

```
================================================================================
================
*/

-- 🟢 STEP 1: CHECK FOR INCORRECT OR NULL VALUES IN `sls_order_dt`
-- ---------------------------------------------------------------
-- 🎯 **Why?** Ensure no invalid or zero values exist in `sls_order_dt`.
-- ✅ **Expectation:** Query should return **only problematic values**.

SELECT
    sls_order_dt
FROM bronze.crm_sales_details
WHERE sls_order_dt <= 0;

-- 🟢 STEP 2: NULLIFY INVALID `sls_order_dt` VALUES
-- ------------------------------------------------
-- 🎯 **Why?** Replace `0` values with `NULL` for consistency.
-- ✅ **Expectation:** Query should return cleaned data.

SELECT
    NULLIF(sls_order_dt, 0) AS sls_order_dt
FROM bronze.crm_sales_details
WHERE sls_order_dt <= 0;

-- 🟢 STEP 3: VALIDATE SALES DATA CONSISTENCY
-- ------------------------------------------
-- 🎯 **Why?** Ensure `sls_sales = sls_quantity * sls_price` and no invalid values exist.
-- ✅ **Expectation:** Query should return **only inconsistent rows**.

SELECT
    sls_sales,
    sls_quantity,
    sls_price
FROM bronze.crm_sales_details
WHERE sls_sales != sls_quantity * sls_price
   OR sls_sales IS NULL OR sls_quantity IS NULL OR sls_price IS NULL
   OR sls_sales <= 0 OR sls_quantity <= 0 OR sls_price <= 0
ORDER BY sls_sales DESC;

-- 🟢 STEP 4: TRANSFORM & CLEAN DATA
-- ---------------------------------
-- 🎯 **Why?** Convert `sls_order_dt`, `sls_ship_dt`, and `sls_due_dt` to correct formats.
-- 🔷 **Handle invalid values (`0 → NULL`)**
-- 🔷 **Ensure sales calculations are correct**

SELECT
    sls_ord_num,
    sls_prd_key,
    sls_cust_id,

    -- ✅ Convert order date to correct format
    CASE
```

```
        WHEN sls_order_dt = 0 OR LENGTH(sls_order_dt::TEXT) != 8 THEN NULL
        ELSE TO_DATE(sls_order_dt::TEXT, 'YYYYMMDD')
    END AS sls_order_dt,

    -- ✅ Convert shipping date to correct format
    CASE
        WHEN sls_ship_dt = 0 OR LENGTH(sls_ship_dt::TEXT) != 8 THEN NULL
        ELSE TO_DATE(sls_ship_dt::TEXT , 'YYYYMMDD')
    END AS sls_ship_dt,

    -- ✅ Convert due date to correct format
    CASE
        WHEN sls_due_dt = 0 OR LENGTH(sls_due_dt::TEXT) != 8 THEN NULL
        ELSE TO_DATE(sls_due_dt::TEXT , 'YYYYMMDD')
    END AS sls_due_dt,

    -- ✅ Correct `sls_sales` if it's inconsistent
    CASE
        WHEN sls_sales IS NULL OR sls_sales <= 0 OR sls_sales != sls_quantity * ABS(sls_price)
            THEN sls_quantity * ABS(sls_price)
        ELSE sls_sales
    END AS sls_sales,

    sls_quantity,

    -- ✅ Fix `sls_price` if it's missing or invalid
    CASE
        WHEN sls_price IS NULL OR sls_price <= 0
            THEN sls_sales / NULLIF(sls_quantity, 0)  -- Prevents division by zero
        ELSE sls_price
    END AS sls_price

FROM bronze.crm_sales_details;

-- 🟢 STEP 5: CREATE THE SILVER TABLE
-- ----------------------------------
-- 🎯 **Why?** Prepare a clean and structured table in the Silver Layer.

DROP TABLE IF EXISTS silver.crm_sales_details;
CREATE TABLE silver.crm_sales_details (
    sls_ord_num VARCHAR(50),   -- Sales Order Number
    sls_prd_key VARCHAR(50),   -- Product Key (Linked to `silver.crm_prd_info`)
    sls_cust_id INT,           -- Customer ID (Linked to `silver.crm_cust_info`)
    sls_order_dt DATE,         -- Order Date (Converted from INT)
    sls_ship_dt DATE,          -- Shipping Date
    sls_due_dt DATE,           -- Due Date for Payment
    sls_sales INT,             -- Total Sales Amount
    sls_quantity INT,          -- Quantity Sold
    sls_price INT,             -- Product Price at the Time of Sale
    dwh_create_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP  -- Data Warehouse Load
Time
);

-- 🟢 STEP 6: INSERT CLEANED DATA INTO THE SILVER TABLE
-- ----------------------------------------------------
-- 🎯 **Why?** Move the cleaned and structured data into the Silver Layer.
```

```
TRUNCATE TABLE silver.crm_sales_details
INSERT INTO silver.crm_sales_details(
    sls_ord_num,
    sls_prd_key,
    sls_cust_id,
    sls_order_dt,
    sls_ship_dt,
    sls_due_dt,
    sls_sales,
    sls_quantity,
    sls_price
)
SELECT
    sls_ord_num,
    sls_prd_key,
    sls_cust_id,

    -- ✅ Convert dates properly
    CASE
        WHEN sls_order_dt = 0 OR LENGTH(sls_order_dt::TEXT) != 8 THEN NULL
        ELSE TO_DATE(sls_order_dt::TEXT, 'YYYYMMDD')
    END AS sls_order_dt,

    CASE
        WHEN sls_ship_dt = 0 OR LENGTH(sls_ship_dt::TEXT) != 8 THEN NULL
        ELSE TO_DATE(sls_ship_dt::TEXT , 'YYYYMMDD')
    END AS sls_ship_dt,

    CASE
        WHEN sls_due_dt = 0 OR LENGTH(sls_due_dt::TEXT) != 8 THEN NULL
        ELSE TO_DATE(sls_due_dt::TEXT , 'YYYYMMDD')
    END AS sls_due_dt,

    -- ✅ Fix sales amount inconsistencies
    CASE
        WHEN sls_sales IS NULL OR sls_sales <= 0 OR sls_sales != sls_quantity * ABS(sls_price)
            THEN sls_quantity * ABS(sls_price)
        ELSE sls_sales
    END AS sls_sales,

    sls_quantity,

    -- ✅ Fix price inconsistencies
    CASE
        WHEN sls_price IS NULL OR sls_price <= 0
            THEN sls_sales / NULLIF(sls_quantity, 0)
        ELSE sls_price
    END AS sls_price

FROM bronze.crm_sales_details;

-- 🟢 STEP 7: VERIFY INSERTED DATA
-- ------------------------------
-- 🎯 **Why?** Ensure the Silver Layer contains the cleaned and transformed data.

SELECT * FROM silver.crm_sales_details;

/*
```

==========
==========

🏛️ **FINAL SUMMARY: SALES DETAILS DATA PROCESSING (`crm_sales_details`)**

==========
==========

🎯 **Goal:**

✅ Ensure data quality by handling **NULL values**, **date inconsistencies**, and **sales mismatches**.

✅ Load **cleaned and structured data** into `silver.crm_sales_details` for analytics.

Step	Action	Expected Outcome
1️⃣	Check NULL & invalid dates	Identifies incorrect date values
2️⃣	Fix invalid `sls_order_dt`	Replaces `0` values with `NULL`
3️⃣	Validate sales calculations	Ensures correct sales values
4️⃣	Transform & clean data	Fixes dates, sales, and price issues
5️⃣	Create Silver table	Prepares structure for clean data
6️⃣	Insert cleaned data	Loads data into Silver Layer
7️⃣	Verify inserted data	Ensures correctness of the data

✅ **Final Check:** After executing all steps, the Silver Layer should have:
- Clean, **accurate sales records**
- No **NULL or invalid dates**
- Correctly formatted **sales calculations**

🎯 **Next Step:** Move validated data to the **Gold Layer** for Business Intelligence processing.

==========
==========
*/

/*
==========
==========

🏛️ **ERP CUSTOMER DATA PROCESSING (`erp_cust_az12`)**

==========
==========

🎯 **Goal:**

✅ Extract, clean, and transform customer data from `bronze.erp_cust_az12`.

✅ Ensure **data consistency** by handling **invalid dates**, **gender standardization**, and **ensuring correct `cid` formatting for table joins**.

✅ Load the **cleaned data** into the Silver Layer (`silver.erp_cust_az12`).

📌 **Steps Overview:**
1️⃣ **Review raw customer data from the Bronze Layer**
2️⃣ **Clean and standardize `cid`, `bdate`, and `gen` columns**
3️⃣ **Identify and handle invalid birthdates (`bdate`)**
4️⃣ **Ensure gender values (`gen`) are correctly formatted**
5️⃣ **Validate transformations and prepare for table joins**
6️⃣ **Create and populate the Silver Table**
7️⃣ **Verify the inserted data**

==========
==========
*/

-- 🟢 STEP 1: REVIEW RAW DATA
-- -------------------------
-- 🎯 **Why?** Inspect the `erp_cust_az12` table before applying transformations.
-- ✅ **Expectation:** Retrieve all raw records for initial review.

SELECT * FROM bronze.erp_cust_az12;

-- 🟢 STEP 2: TRANSFORM `cid`, `bdate`, AND `gen`
-- ---
-- 🎯 **Why?** Standardize `cid`, ensure valid birthdates, and clean gender values.
-- ✅ **Transformations Applied:**
--   🔷 Remove 'NAS' prefix from `cid` to enable table joins.
--   🔷 Convert invalid birthdates (`bdate > TODAY`) to NULL.
--   🔷 Standardize `gen` values to 'Male', 'Female', or 'Other'.

```
SELECT
    -- ✅ Remove 'NAS' prefix from `cid` (if it exists)
    CASE
        WHEN cid LIKE 'NAS%' THEN SUBSTRING(cid, 4, LENGTH(cid))
        ELSE cid
    END AS cid,

    -- ✅ Nullify future birthdates (they are invalid)
    CASE
        WHEN bdate > CURRENT_DATE THEN NULL
        ELSE bdate
    END AS bdate,

    -- ✅ Standardize gender values
    CASE
        WHEN UPPER(TRIM(gen)) IN ('F', 'FEMALE') THEN 'Female'
        WHEN UPPER(TRIM(gen)) IN ('M', 'MALE') THEN 'Male'
        ELSE 'Other'
    END AS gen
```

```
FROM bronze.erp_cust_az12;

-- 🟢 STEP 3: IDENTIFY INCORRECT BIRTHDATES (`bdate`)
-- ------------------------------------------------
-- 🎯 **Why?** Identify customers with **extreme birthdates** (before 1924 or future dates).
-- ✅ **Expectation:** This should return **only problematic records**.

SELECT *
FROM bronze.erp_cust_az12
WHERE bdate < '1924-01-01' OR bdate > CURRENT_DATE;

-- 🟢 STEP 4: CHECK UNIQUE VALUES IN `gen` COLUMN
-- ----------------------------------------------
-- 🎯 **Why?** Verify **all unique gender values** to ensure proper standardization.
-- ✅ **Expectation:** The query should return only expected values ('Male', 'Female', 'Other').

SELECT DISTINCT(gen)
FROM bronze.erp_cust_az12;

-- 🟢 STEP 5: CREATE THE SILVER TABLE
-- ----------------------------------
-- 🎯 **Why?** Prepare a clean and structured table in the Silver Layer.

DROP TABLE IF EXISTS silver.erp_cust_az12;
CREATE TABLE silver.erp_cust_az12 (
    cid VARCHAR(50),      -- ✅ Customer ID (Standardized)
    bdate DATE,           -- ✅ Birthdate (Validated)
    gen VARCHAR(50)       -- ✅ Gender (Standardized)
);

-- 🟢 STEP 6: INSERT CLEANED DATA INTO THE SILVER TABLE
-- ----------------------------------------------------
-- 🎯 **Why?** Load transformed data into the Silver Layer for analytics.
TRUNCATE TABLE silver.erp_cust_az12
INSERT INTO silver.erp_cust_az12 (
    cid,      -- Customer ID
    bdate,    -- Birthdate
    gen       -- Gender
)
SELECT
    -- ✅ Remove 'NAS' prefix from `cid`
    CASE
        WHEN cid LIKE 'NAS%' THEN SUBSTRING(cid, 4, LENGTH(cid))
        ELSE cid
    END AS cid,

    -- ✅ Nullify future birthdates
    CASE
        WHEN bdate > CURRENT_DATE THEN NULL
        ELSE bdate
    END AS bdate,
```

```
    -- ✅ Standardize gender values
    CASE
        WHEN UPPER(TRIM(gen)) IN ('F', 'FEMALE') THEN 'Female'
        WHEN UPPER(TRIM(gen)) IN ('M', 'MALE') THEN 'Male'
        ELSE 'Other'
    END AS gen

FROM bronze.erp_cust_az12;

-- 🟢 STEP 7: VERIFY INSERTED DATA
-- ------------------------------
-- 🎯 **Why?** Ensure that data in the Silver Layer is correctly formatted.

SELECT * FROM silver.erp_cust_az12;

/*
===============================================================================
🏛️ **FINAL SUMMARY: ERP CUSTOMER DATA PROCESSING (`erp_cust_az12`)**
===============================================================================

🎯 **Goal:**
✅ Ensure data quality by handling **NULL values**, **date inconsistencies**, and **gender standardization**.
✅ Load **cleaned and structured data** into `silver.erp_cust_az12` for further analytics.

┌──────────┬──────────────────────────┬──────────────────────────────────────┐
| Step     |        Action            |          Expected Outcome            |
├──────────┼──────────────────────────┼──────────────────────────────────────┤
| 1️⃣       | Review raw data          | Retrieve all customer records        |
| 2️⃣       | Transform `cid`, `bdate`, `gen` | Standardized values for consistency |
| 3️⃣       | Identify incorrect birthdates | Detect extreme/future birthdates |
| 4️⃣       | Check gender values      | Ensure only 'Male', 'Female', 'Other' |
| 5️⃣       | Create Silver table      | Prepare structure for clean data     |
| 6️⃣       | Insert cleaned data      | Load data into Silver Layer          |
| 7️⃣       | Verify inserted data     | Ensure correctness of transformation |
└──────────┴──────────────────────────┴──────────────────────────────────────┘

✅ **Final Check:** After executing all steps, the Silver Layer should have:
  - Clean, **accurate customer records**
  - No **NULL or invalid birthdates**
  - Correctly formatted **gender values**

🎯 **Next Step:** Move validated data to the **Gold Layer** for Business Intelligence processing.
```

```
================================================================================
================
*/

/*
================================================================================
================
🏛️ **ERP LOCATION DATA PROCESSING (`erp_loc_a101`)**
================================================================================
================

🎯 **Goal:**
✅ Extract, clean, and transform location data from `bronze.erp_loc_a101`.
✅ Ensure **data consistency** by:
  - **Standardizing `cid` values** (removing `-` from IDs).
  - **Normalizing country names (`cntry`)** for consistency.
  - **Handling NULL or empty country values**.
✅ Load the **cleaned data** into the Silver Layer (`silver.erp_loc_a101`).

📌 **Steps Overview:**
1️⃣ **Review raw location data from the Bronze Layer**
2️⃣ **Standardize `cid` by removing hyphens (`-`)**
3️⃣ **Transform `cntry` values into standardized country names**
4️⃣ **Identify unique country values (`cntry`) for validation**
5️⃣ **Create and populate the Silver Table**
6️⃣ **Verify the inserted data**

================================================================================
================
*/

-- 🟢 STEP 1: REVIEW RAW DATA
-- -------------------------
-- 🎯 **Why?** Inspect the `erp_loc_a101` table before applying transformations.
-- ✅ **Expectation:** Retrieve all raw records for initial review.

SELECT * FROM bronze.erp_loc_a101;

-- 🟢 STEP 2: TRANSFORM `cid` AND `cntry`
-- --------------------------------------
-- 🎯 **Why?** Standardize `cid` (remove `-`), normalize country values, and handle NULLs.
-- ✅ **Transformations Applied:**
--    🔷 **Remove hyphens (`-`)** from `cid`.
--    🔷 **Convert `DE` to `Germany`**.
--    🔷 **Convert `US`/`USA` to `United States`**.
--    🔷 **Replace empty or NULL country values with `'n/a'`**.

SELECT
```

```
    -- ✅ Remove hyphens from `cid`
    REPLACE(cid, '-', '') AS cid,

    -- ✅ Standardize country names
    CASE
        WHEN cntry = 'DE' THEN 'Germany'
        WHEN TRIM(cntry) IN ('US', 'USA') THEN 'United States'
        WHEN TRIM(cntry) = '' OR cntry IS NULL THEN 'n/a'
        ELSE cntry
    END AS cntry

FROM bronze.erp_loc_a101;

-- 🟢 STEP 3: IDENTIFY UNIQUE COUNTRY VALUES (`cntry`)
-- ---------------------------------------------------
-- 🎯 **Why?** Verify **all unique country values** to ensure proper standardization.
-- ✅ **Expectation:** The query should return all distinct values in `cntry`.

SELECT
    DISTINCT cntry
FROM bronze.erp_loc_a101
GROUP BY cntry;

-- 🟢 STEP 4: CREATE THE SILVER TABLE
-- ----------------------------------
-- 🎯 **Why?** Prepare a clean and structured table in the Silver Layer.

DROP TABLE IF EXISTS silver.erp_loc_a101;
CREATE TABLE silver.erp_loc_a101 (
    cid VARCHAR(50),     -- ✅ Standardized Customer ID
    cntry VARCHAR(50)    -- ✅ Normalized Country Name
);

-- 🟢 STEP 5: INSERT CLEANED DATA INTO THE SILVER TABLE
-- ----------------------------------------------------
-- 🎯 **Why?** Load transformed data into the Silver Layer for analytics.
TRUNCATE TABLE silver.erp_loc_a101
INSERT INTO silver.erp_loc_a101 (
    cid,
    cntry
)
SELECT
    -- ✅ Remove hyphens from `cid`
    REPLACE(cid, '-', '') AS cid,

    -- ✅ Standardize country names
    CASE
        WHEN cntry = 'DE' THEN 'Germany'
        WHEN TRIM(cntry) IN ('US', 'USA') THEN 'United States'
        WHEN TRIM(cntry) = '' OR cntry IS NULL THEN 'n/a'
        ELSE cntry
    END AS cntry
```

FROM bronze.erp_loc_a101;

-- 🟢 STEP 6: VERIFY INSERTED DATA
-- ------------------------------
-- 🎯 **Why?** Ensure that data in the Silver Layer is correctly formatted.

SELECT * FROM silver.erp_loc_a101;

/*
===
===============
🏛️ **FINAL SUMMARY: ERP LOCATION DATA PROCESSING (`erp_loc_a101`)**
===
===============

🎯 **Goal:**
✅ Ensure data consistency by **cleaning `cid` values** and **standardizing country names**.
✅ Load **cleaned and structured data** into `silver.erp_loc_a101` for further analytics.

Step	Action	Expected Outcome
1️⃣	Review raw data	Retrieve all location records
2️⃣	Transform `cid` and `cntry`	Standardized values for consistency
3️⃣	Identify unique countries	Detect inconsistent country names
4️⃣	Create Silver table	Prepare structure for clean data
5️⃣	Insert cleaned data	Load data into Silver Layer
6️⃣	Verify inserted data	Ensure correctness of transformation

✅ **Final Check:** After executing all steps, the Silver Layer should have:
- Clean, **accurate location records**
- No **NULL or incorrect country values**
- Standardized **customer IDs (`cid`)**

🎯 **Next Step:** Move validated data to the **Gold Layer** for Business Intelligence processing.

===
===============
*/

/*
===
===============
🏛️ **ERP PRODUCT CATEGORY DATA PROCESSING (`erp_px_cat_g1v2`)**
===
===============

🎯 **Goal:**

✅ Extract, clean, and transform product category data from `bronze.erp_px_cat_g1v2`.

✅ Ensure **data consistency** by:
  - **Trimming unnecessary spaces** in `cat`, `subcat`, and `maintenance`.
  - **Checking unique values** to detect inconsistencies.

✅ Load the **cleaned data** into the Silver Layer (`silver.erp_px_cat_g1v2`)
  by first truncating the table to remove old data.

📌 **Steps Overview:**

1️⃣ **Review raw product category data from the Bronze Layer**

2️⃣ **Check for unnecessary spaces in key columns**

3️⃣ **Validate distinct values in the `maintenance` column**

4️⃣ **Create the Silver Table (if not exists)**

5️⃣ **Truncate Silver Table before inserting new data**

6️⃣ **Insert cleaned and transformed data**

7️⃣ **Verify the inserted data**

==========
==========
*/

-- 🟢 STEP 1: REVIEW RAW DATA
-- -------------------------
-- 🎯 **Why?** Inspect the `erp_px_cat_g1v2` table before applying transformations.
-- ✅ **Expectation:** Retrieve all raw records for initial review.

```
SELECT
    id,
    cat,
    subcat,
    maintenance
FROM bronze.erp_px_cat_g1v2;
```

-- 🟢 STEP 2: CHECK FOR UNWANTED SPACES
-- -----------------------------------
-- 🎯 **Why?** Ensure `cat`, `subcat`, and `maintenance` values do not have leading/trailing spaces.
-- ✅ **Expectation:** Query should return only rows with unnecessary spaces.

```
SELECT
    *
FROM bronze.erp_px_cat_g1v2
WHERE cat != TRIM(cat) OR subcat != TRIM(subcat) OR maintenance != TRIM(maintenance);
```

-- 🟢 STEP 3: IDENTIFY UNIQUE MAINTENANCE VALUES
-- --
-- 🎯 **Why?** Verify **all unique `maintenance` values** to ensure proper standardization.
-- ✅ **Expectation:** The query should return distinct `maintenance` values.

```
SELECT DISTINCT maintenance
FROM bronze.erp_px_cat_g1v2;

-- 🟢 STEP 4: CREATE THE SILVER TABLE (IF NOT EXISTS)
-- ------------------------------------------------
-- 🎯 **Why?** Prepare a clean and structured table in the Silver Layer.

DROP TABLE IF EXISTS silver.erp_px_cat_g1v2;
CREATE TABLE silver.erp_px_cat_g1v2 (
    id VARCHAR(50),          -- ✅ Product ID
    cat VARCHAR(50),         -- ✅ Category Name (Trimmed)
    subcat VARCHAR(50),      -- ✅ Subcategory Name (Trimmed)
    maintenance VARCHAR(50)  -- ✅ Maintenance Type (Trimmed)
);

-- 🟢 STEP 5: TRUNCATE SILVER TABLE BEFORE INSERTING DATA
-- ------------------------------------------------------
-- 🎯 **Why?** Ensure the Silver table is empty before inserting new data to avoid duplicates.
TRUNCATE TABLE silver.erp_px_cat_g1v2;

-- 🟢 STEP 6: INSERT CLEANED DATA INTO THE SILVER TABLE
-- ----------------------------------------------------
-- 🎯 **Why?** Load transformed data into the Silver Layer for analytics.
-- ✅ **Transformations Applied:**
--    🔷 **Trim spaces** from `cat`, `subcat`, and `maintenance`.
TRUNCATE TABLE silver.erp_px_cat_g1v2
INSERT INTO silver.erp_px_cat_g1v2 (
    id,
    cat,
    subcat,
    maintenance
)
SELECT
    id,
    TRIM(cat) AS cat,          -- ✅ Ensure no spaces
    TRIM(subcat) AS subcat,    -- ✅ Ensure no spaces
    TRIM(maintenance) AS maintenance  -- ✅ Ensure no spaces
FROM bronze.erp_px_cat_g1v2;

-- 🟢 STEP 7: VERIFY INSERTED DATA
-- -------------------------------
-- 🎯 **Why?** Ensure that data in the Silver Layer is correctly formatted.

SELECT * FROM silver.erp_px_cat_g1v2;

/*
===============================================================================
================
🏛️ **FINAL SUMMARY: ERP PRODUCT CATEGORY DATA PROCESSING (`erp_px_cat_g1v2`)**
```

==========

🎯 **Goal:**

✅ Ensure data consistency by **removing unnecessary spaces** and **standardizing product category values**.

✅ Load **cleaned and structured data** into `silver.erp_px_cat_g1v2` for further analytics.

Step	Action	Expected Outcome
1️⃣	Review raw data	Retrieve all product category records
2️⃣	Check for unwanted spaces	Detect leading/trailing spaces
3️⃣	Identify unique maintenance	Ensure standardized values
4️⃣	Create Silver table	Prepare structure for clean data
5️⃣	Insert cleaned data	Load data into Silver Layer
6️⃣	Verify inserted data	Ensure correctness of transformation

✅ **Final Check:** After executing all steps, the Silver Layer should have:
- Clean, **accurate product category records**
- No **NULL or incorrect maintenance values**
- Standardized **category and subcategory names**

🎯 **Next Step:** Move validated data to the **Gold Layer** for Business Intelligence processing.

==========
*/

/*
==========

🏛️ **GOLD LAYER: CUSTOMER DIMENSION (`dim_customers`)**

==========

🎯 **Goal:**

✅ Transform and enrich customer data from the **Silver Layer** into the **Gold Layer**.

✅ Ensure **data consistency** by:
- **Merging CRM and ERP customer data** (gender, birth date, location).
- **Checking for duplicates introduced by joins**.
- **Renaming columns for readability**.
- **Creating a View (`gold.dim_customers`) with a surrogate key**.

📌 **Steps Overview:**

1️⃣ **Review Silver Layer Customer Data**
2️⃣ **Perform Customer Data Enrichment** (CRM + ERP integration)
3️⃣ **Check for Duplicates After Join**
4️⃣ **Validate Gender Standardization**
5️⃣ **Standardize Column Names for Readability**
6️⃣ **Create `gold.dim_customers` View with Surrogate Key**
7️⃣ **Verify Gold Layer Transformation**

==========
*/

-- 🟢 STEP 1: REVIEW SILVER LAYER CUSTOMER DATA
-- --
-- 🎯 **Why?** Check the raw customer data before transformation.
-- ✅ **Expectation:** Retrieve all customers from `silver.crm_cust_info`.

SELECT * FROM silver.crm_cust_info;

-- 🟢 STEP 2: PERFORM CUSTOMER DATA ENRICHMENT (JOIN CRM & ERP)
-- --
-- 🎯 **Why?** Merge customer data from CRM (`crm_cust_info`), ERP (`erp_cust_az12`), and Location (`erp_loc_a101`).
-- ✅ **Transformations Applied:**
--   🔷 **Gender standardization**: Use ERP gender when CRM says "Other".
--   🔷 **Birthdate enrichment**: Add `bdate` from ERP.
--   🔷 **Location mapping**: Add `cntry` from ERP Location.

SELECT
    ci.cst_id,
    ci.cst_key,
    ci.cst_firstname,
    ci.cst_lastname,
    ci.cst_marital_status,
    -- ✅ Use ERP gender if CRM gender is 'Other'
    CASE
        WHEN ci.cst_gndr = 'Other' THEN ca.gen
        ELSE ci.cst_gndr
    END AS gender,
    ci.cst_create_date,
    ca.bdate,
    la.cntry
FROM silver.crm_cust_info ci
LEFT JOIN silver.erp_cust_az12 ca ON ci.cst_key = ca.cid
LEFT JOIN silver.erp_loc_a101 la ON ci.cst_key = la.cid;

-- 🟢 STEP 3: CHECK FOR DUPLICATES AFTER JOIN
-- --
-- 🎯 **Why?** Ensure that the join does not introduce duplicate customers.
-- ✅ **Expectation:** Query should return **0 rows** if no duplicates exist.

```
SELECT cst_id, COUNT(*)
FROM (
    SELECT
        ci.cst_id,
        ci.cst_key,
        ci.cst_firstname,
        ci.cst_lastname,
        ci.cst_marital_status,
        ci.cst_gndr,
        ci.cst_create_date,
        ca.bdate,
        ca.gen,
        la.cntry
    FROM silver.crm_cust_info ci
    LEFT JOIN silver.erp_cust_az12 ca ON ci.cst_key = ca.cid
    LEFT JOIN silver.erp_loc_a101 la ON ci.cst_key = la.cid
) t
GROUP BY cst_id
HAVING COUNT(*) > 1;

-- 🟢 STEP 4: VALIDATE GENDER STANDARDIZATION
-- ------------------------------------------
-- 🎯 **Why?** Ensure that gender is correctly standardized based on CRM & ERP data.
-- ✅ **Expectation:** The `new_gender` column should contain correct gender values.

SELECT
    ci.cst_gndr AS crm_gender,
    ca.gen AS erp_gender,
    CASE
        WHEN ci.cst_gndr = 'Other' THEN ca.gen
        ELSE ci.cst_gndr
    END AS new_gender
FROM silver.crm_cust_info ci
LEFT JOIN silver.erp_cust_az12 ca ON ci.cst_key = ca.cid
LEFT JOIN silver.erp_loc_a101 la ON ci.cst_key = la.cid;

-- 🟢 STEP 5: STANDARDIZE COLUMN NAMES FOR READABILITY
-- ---------------------------------------------------
-- 🎯 **Why?** Improve table usability with **business-friendly column names**.
-- ✅ **Transformations Applied:**
--    🔷 `cst_id` → `customer_id`
--    🔷 `cst_key` → `customer_number`
--    🔷 `cst_firstname` → `first_name`
--    🔷 `cst_lastname` → `last_name`
--    🔷 `cst_marital_status` → `marital_status`
--    🔷 `cst_gndr` → `gender`
--    🔷 `cst_create_date` → `create_date`

SELECT
    ci.cst_id AS customer_id,
    ci.cst_key AS customer_number,
    ci.cst_firstname AS first_name,
```

```
    ci.cst_lastname AS last_name,
    la.cntry AS country,
    ci.cst_marital_status AS marital_status,
    -- ✅ Use ERP gender if CRM gender is 'Other'
    CASE
        WHEN ci.cst_gndr = 'Other' THEN ca.gen
        ELSE ci.cst_gndr
    END AS gender,
    ca.bdate AS birth_date,
    ci.cst_create_date AS create_date
FROM silver.crm_cust_info ci
LEFT JOIN silver.erp_cust_az12 ca ON ci.cst_key = ca.cid
LEFT JOIN silver.erp_loc_a101 la ON ci.cst_key = la.cid;

-- 🟢 STEP 6: CREATE GOLD LAYER VIEW (`dim_customers`)
-- ------------------------------------------------
-- 🎯 **Why?** Create a **Gold Layer Dimension Table** with a **Surrogate Key (`customer_key`)**.
-- ✅ **Transformations Applied:**
--    🔷 Add **`customer_key`** (generated using `ROW_NUMBER() OVER (ORDER BY cst_id)`).
--    🔷 Use **cleaned and standardized customer data**.

CREATE VIEW gold.dim_customers AS
SELECT
    ROW_NUMBER() OVER (ORDER BY cst_id) AS customer_key,  -- ✅ Surrogate Key
    ci.cst_id AS customer_id,
    ci.cst_key AS customer_number,
    ci.cst_firstname AS first_name,
    ci.cst_lastname AS last_name,
    la.cntry AS country,
    ci.cst_marital_status AS marital_status,
    -- ✅ Use ERP gender if CRM gender is 'Other'
    CASE
        WHEN ci.cst_gndr = 'Other' THEN ca.gen
        ELSE ci.cst_gndr
    END AS gender,
    ca.bdate AS birth_date,
    ci.cst_create_date AS create_date
FROM silver.crm_cust_info ci
LEFT JOIN silver.erp_cust_az12 ca ON ci.cst_key = ca.cid
LEFT JOIN silver.erp_loc_a101 la ON ci.cst_key = la.cid;

-- 🟢 STEP 7: VERIFY GOLD LAYER TRANSFORMATION
-- -------------------------------------------
-- 🎯 **Why?** Ensure that the **Gold Layer** (`dim_customers`) contains **clean, structured data**.

SELECT * FROM gold.dim_customers;

/*
===============================================================================
🏛️ **FINAL SUMMARY: GOLD LAYER CUSTOMER DIMENSION (`dim_customers`)**
===============================================================================
```

🎯 **Goal:**
✅ **Enrich CRM customer data** by integrating ERP information.
✅ **Standardize customer attributes** (gender, birth date, country).
✅ **Ensure no duplicate customers exist after merging data**.
✅ **Create a Gold Layer (`dim_customers`) with a **Surrogate Key**.

Step	Action	Expected Outcome
1️⃣	Review Silver data	Retrieve all CRM customers
2️⃣	Perform data enrichment	Merge CRM & ERP customer data
3️⃣	Check for duplicates	Ensure unique customer records
4️⃣	Validate gender consistency	Standardize gender values
5️⃣	Standardize column names	Improve table readability
6️⃣	Create Gold Layer view	Store cleaned customer data
7️⃣	Verify Gold data	Ensure correctness of transformation

==========
*/

/*
==========
🏛️ **GOLD LAYER: PRODUCT DIMENSION (`dim_products`)**
==========

🎯 **Goal:**
✅ Transform and enrich product data from the **Silver Layer** into the **Gold Layer**.
✅ Ensure **data consistency** by:
- **Merging CRM product data with ERP category data**.
- **Filtering out historical (inactive) products**.
- **Checking for duplicates introduced by joins**.
- **Renaming columns for readability**.
- **Creating a View (`gold.dim_products`) with a surrogate key**.

📌 **Steps Overview:**
1️⃣ **Review Silver Layer Product Data**
2️⃣ **Perform Product Data Enrichment** (CRM + ERP integration)
3️⃣ **Filter Out Historical (Inactive) Products**

4️⃣ **Check for Duplicates After Join**
5️⃣ **Standardize Column Names for Readability**
6️⃣ **Create `gold.dim_products` View with Surrogate Key**
7️⃣ **Verify Gold Layer Transformation**

==========
*/

-- 🟢 STEP 1: REVIEW SILVER LAYER PRODUCT DATA
-- --
-- 🎯 **Why?** Check the raw product data before transformation.
-- ✅ **Expectation:** Retrieve all products from `silver.crm_prd_info`.

SELECT * FROM silver.crm_prd_info;

-- 🟢 STEP 2: PERFORM PRODUCT DATA ENRICHMENT (JOIN CRM & ERP)
-- --
-- 🎯 **Why?** Merge product data from CRM (`crm_prd_info`) and ERP category details (`erp_px_cat_g1v2`).
-- ✅ **Transformations Applied:**
--   🔷 **Category Mapping**: Join `cat_id` with ERP categories.
--   🔷 **Filter Only Active Products**: Exclude historical data where `prd_end_dt IS NULL`.

```
SELECT
    pn.prd_id,
    pn.cat_id,
    pn.prd_key,
    pn.prd_nm,
    pn.prd_cost,
    pn.prd_line,
    pn.prd_start_dt,
    pc.cat,         -- ✅ Category from ERP
    pc.subcat,      -- ✅ Subcategory from ERP
    pc.maintenance  -- ✅ Maintenance from ERP
FROM silver.crm_prd_info pn
LEFT JOIN silver.erp_px_cat_g1v2 pc ON pn.cat_id = pc.id
WHERE prd_end_dt IS NULL;  -- ✅ Keep only active products
```

-- 🟢 STEP 3: CHECK FOR DUPLICATES AFTER JOIN
-- --
-- 🎯 **Why?** Ensure that the join does not introduce duplicate products.
-- ✅ **Expectation:** Query should return **0 rows** if no duplicates exist.

```
SELECT prd_key, COUNT(*)
FROM (
    SELECT
        pn.prd_id,
        pn.cat_id,
        pn.prd_key,
```

```
        pn.prd_nm,
        pn.prd_cost,
        pn.prd_line,
        pn.prd_start_dt,
        pc.cat,
        pc.subcat,
        pc.maintenance
    FROM silver.crm_prd_info pn
    LEFT JOIN silver.erp_px_cat_g1v2 pc ON pn.cat_id = pc.id
    WHERE prd_end_dt IS NULL
) t
GROUP BY prd_key
HAVING COUNT(*) > 1;

-- 🟢 STEP 4: STANDARDIZE COLUMN NAMES FOR READABILITY
-- ------------------------------------------------
-- 🎯 **Why?** Improve table usability with **business-friendly column names**.
-- ✅ **Transformations Applied:**
--    🔷 `prd_id` → `product_id`
--    🔷 `prd_key` → `product_number`
--    🔷 `prd_nm` → `product_name`
--    🔷 `cat_id` → `category_id`
--    🔷 `prd_cost` → `cost`
--    🔷 `prd_line` → `product_line`
--    🔷 `prd_start_dt` → `start_date`

SELECT
    pn.prd_id AS product_id,
    pn.prd_key AS product_number,
    pn.prd_nm AS product_name,
    pn.cat_id AS category_id,
    pc.cat AS category,
    pc.subcat AS subcategory,
    pc.maintenance,
    pn.prd_cost AS cost,
    pn.prd_line AS product_line,
    pn.prd_start_dt AS start_date
FROM silver.crm_prd_info pn
LEFT JOIN silver.erp_px_cat_g1v2 pc ON pn.cat_id = pc.id
WHERE prd_end_dt IS NULL;

-- 🟢 STEP 5: CREATE GOLD LAYER VIEW (`dim_products`)
-- ------------------------------------------------
-- 🎯 **Why?** Create a **Gold Layer Dimension Table** with a **Surrogate Key (`product_key`)**.
-- ✅ **Transformations Applied:**
--    🔷 Add **`product_key`** (generated using `ROW_NUMBER() OVER (ORDER BY prd_start_dt, prd_key)`).
--    🔷 Use **cleaned and standardized product data**.

CREATE VIEW gold.dim_products AS
SELECT
```

```
    ROW_NUMBER() OVER (ORDER BY pn.prd_start_dt, pn.prd_key) AS product_key,  -- ✅ Surrogate Key
    pn.prd_id AS product_id,
    pn.prd_key AS product_number,
    pn.prd_nm AS product_name,
    pn.cat_id AS category_id,
    pc.cat AS category,
    pc.subcat AS subcategory,
    pc.maintenance,
    pn.prd_cost AS cost,
    pn.prd_line AS product_line,
    pn.prd_start_dt AS start_date
FROM silver.crm_prd_info pn
LEFT JOIN silver.erp_px_cat_g1v2 pc ON pn.cat_id = pc.id
WHERE prd_end_dt IS NULL;

-- 🟢 STEP 6: VERIFY GOLD LAYER TRANSFORMATION
-- ------------------------------------------
-- 🎯 **Why?** Ensure that the **Gold Layer** (`dim_products`) contains **clean, structured data**.

SELECT * FROM gold.dim_products;

/*
===============================================================================
🏛️ **FINAL SUMMARY: GOLD LAYER PRODUCT DIMENSION (`dim_products`)**
===============================================================================

🎯 **Goal:**
✅ **Enrich CRM product data** by integrating ERP category details.
✅ **Standardize product attributes** (category, subcategory, maintenance).
✅ **Ensure no duplicate products exist after merging data**.
✅ **Create a Gold Layer (`dim_products`) with a **Surrogate Key**.

┌──────────┬──────────────────────┬──────────────────────────────┐
│ Step     │ Action               │ Expected Outcome             │
├──────────┼──────────────────────┼──────────────────────────────┤
│ 1️⃣       │ Review Silver data   │ Retrieve all CRM products    │
│ 2️⃣       │ Perform data enrichment │ Merge CRM & ERP product data │
│ 3️⃣       │ Filter active products │ Exclude inactive product records │
│ 4️⃣       │ Check for duplicates │ Ensure unique product records │
│ 5️⃣       │ Standardize column names │ Improve table readability │
│ 6️⃣       │ Create Gold Layer view │ Store cleaned product data │
│ 7️⃣       │ Verify Gold data     │ Ensure correctness of transformation │
└──────────┴──────────────────────┴──────────────────────────────┘
```

```
================================================================================
================
*/


/*
================================================================================
================
🏛️ **GOLD LAYER: SALES FACT TABLE (`fact_sales`)**
================================================================================
================

🎯 **Goal:**
✅ Transform and enrich sales transaction data from the **Silver Layer** into the **Gold Layer**.
✅ Ensure **data consistency** by:
  - **Joining Sales data with Product and Customer Dimensions**.
  - **Creating a central Fact Table (`fact_sales`) for analytics**.
  - **Verifying successful joins between Fact and Dimension tables**.

📌 **Steps Overview:**
1️⃣ **Review Silver Layer Sales Data**
2️⃣ **Create `gold.fact_sales` View by Joining Sales, Products, and Customers**
3️⃣ **Validate Gold Layer Data (`fact_sales`)**
4️⃣ **Check if Dimension Tables Join Successfully**

================================================================================
================
*/

-- 🟢 STEP 1: REVIEW SILVER LAYER SALES DATA
-- ----------------------------------------
-- 🎯 **Why?** Check the raw sales data before transformation.
-- ✅ **Expectation:** Retrieve all sales transactions from `silver.crm_sales_details`.

SELECT * FROM silver.crm_sales_details;

-- 🟢 STEP 2: CREATE GOLD LAYER FACT TABLE (`fact_sales`)
-- ----------------------------------------------------
-- 🎯 **Why?** Create a **Fact Table** to centralize sales transactions.
-- ✅ **Transformations Applied:**
--    🔷 **Join with Product Dimension** (`dim_products`) on `sls_prd_key`.
--    🔷 **Join with Customer Dimension** (`dim_customers`) on `sls_cust_id`.
--    🔷 **Rename columns for readability**.

CREATE VIEW gold.fact_sales AS
SELECT
    sd.sls_ord_num AS order_number,
    pr.product_key,
    cu.customer_key,
    sd.sls_order_dt AS order_date,
```

```
    sd.sls_ship_dt AS shipping_date,
    sd.sls_due_dt AS due_date,
    sd.sls_sales AS sales_amount,
    sd.sls_quantity AS quantity,
    sd.sls_price AS price
FROM silver.crm_sales_details sd
LEFT JOIN gold.dim_products pr ON sd.sls_prd_key = pr.product_number
LEFT JOIN gold.dim_customers cu ON sd.sls_cust_id = cu.customer_id;

-- 🟢 STEP 3: VERIFY GOLD LAYER FACT TABLE (`fact_sales`)
-- ------------------------------------------------------
-- 🎯 **Why?** Ensure that the **Gold Layer** (`fact_sales`) contains **clean, structured data**.

SELECT * FROM gold.fact_sales;

-- 🟢 STEP 4: VALIDATE JOINED DIMENSIONS IN `fact_sales`
-- ------------------------------------------------------
-- 🎯 **Why?** Ensure that all sales records have valid Product and Customer references.
-- ✅ **Expectation:** Query should return **0 rows** if all joins are successful.

SELECT * FROM gold.fact_sales f
LEFT JOIN gold.dim_customers c ON c.customer_key = f.customer_key
LEFT JOIN gold.dim_products p ON p.product_key = f.product_key
WHERE p.product_key IS NULL;  -- ✅ Detect missing product references

-- 🟢 STEP 5: VALIDATE CUSTOMER DIMENSION TABLE
-- ---------------------------------------------
-- 🎯 **Why?** Ensure customer records are correctly mapped.
```

2. gold.dim_products

- **Purpose:** Provides information about the products and their attributes.
- **Columns:**

Column Name	Data Type	Description
product_key	INT	Surrogate key uniquely identifying each product record in the product dimension table.
product_id	INT	A unique identifier assigned to the product for internal tracking and referencing.
product_number	NVARCHAR(50)	A structured alphanumeric code representing the product, often used for categorization or inventory.
product_name	NVARCHAR(50)	Descriptive name of the product, including key details such as type, color, and size.
category_id	NVARCHAR(50)	A unique identifier for the product's category, linking to its high-level classification.
category	NVARCHAR(50)	The broader classification of the product (e.g., Bikes, Components) to group related items.
subcategory	NVARCHAR(50)	A more detailed classification of the product within the category, such as product type.
maintenance_required	NVARCHAR(50)	Indicates whether the product requires maintenance (e.g., 'Yes', 'No').
cost	INT	The cost or base price of the product, measured in monetary units.
product_line	NVARCHAR(50)	The specific product line or series to which the product belongs (e.g., Road, Mountain).
start_date	DATE	The date when the product became available for sale or use, stored in

3. gold.fact_sales

- **Purpose:** Stores transactional sales data for analytical purposes.
- **Columns:**

Column Name	Data Type	Description
order_number	NVARCHAR(50)	A unique alphanumeric identifier for each sales order (e.g., 'SO54496').
product_key	INT	Surrogate key linking the order to the product dimension table.
customer_key	INT	Surrogate key linking the order to the customer dimension table.
order_date	DATE	The date when the order was placed.
shipping_date	DATE	The date when the order was shipped to the customer.
due_date	DATE	The date when the order payment was due.
sales_amount	INT	The total monetary value of the sale for the line item, in whole currency units (e.g., 25).
quantity	INT	The number of units of the product ordered for the line item (e.g., 1).
price	INT	The price per unit of the product for the line item, in whole currency units (e.g., 25).

-- ✅ **Expectation:** Retrieve all customer records.

SELECT * FROM gold.dim_customers;

/*
==========
==========
🏛️ **FINAL SUMMARY: GOLD LAYER SALES FACT TABLE (`fact_sales`)**
==========
==========

🎯 **Goal:**
✅ **Centralize Sales Data** by joining CRM transactions with Product and Customer Dimensions.
✅ **Standardize Fact Table attributes** (Order Date, Amount, Quantity, Price).
✅ **Ensure Dimension Tables successfully join to the Fact Table**.

Step	Action	Expected Outcome
1️⃣	Review Silver Sales data	Retrieve all CRM sales transactions
2️⃣	Create `fact_sales` view	Join Sales with Product & Customers
3️⃣	Verify Gold Fact Table	Ensure correctly structured data

Data Catalog for Gold Layer

Overview

The Gold Layer is the business-level data representation, structured to support analytical and reporting use cases. It consists of **dimension tables** and **fact tables** for specific business metrics.

---

1. gold.dim_customers

- **Purpose:** Stores customer details enriched with demographic and geographic data.
- **Columns:**

Column Name	Data Type	Description
customer_key	INT	Surrogate key uniquely identifying each customer record in the dimension table.
customer_id	INT	Unique numerical identifier assigned to each customer.
customer_number	NVARCHAR(50)	Alphanumeric identifier representing the customer, used for tracking and referencing.
first_name	NVARCHAR(50)	The customer's first name, as recorded in the system.
last_name	NVARCHAR(50)	The customer's last name or family name.
country	NVARCHAR(50)	The country of residence for the customer (e.g., 'Australia').
marital_status	NVARCHAR(50)	The marital status of the customer (e.g., 'Married', 'Single').
gender	NVARCHAR(50)	The gender of the customer (e.g., 'Male', 'Female', 'n/a').
birthdate	DATE	The date of birth of the customer, formatted as YYYY-MM-DD (e.g., 1971-10-06).
create_date	DATE	The date and time when the customer record was created in the system

| 4️⃣ | Check Dimension Joins | Ensure no missing Product references |
| 5️⃣ | Validate Customers | Retrieve all Customers successfully |

==========
==========
*/

Data Analysis

BY *www.linkedin.com/in/mj-sharifi*